

Response to Ball's Final-Review Strategic and Architectural Questions

Recommendations, rationale and rejected alternatives for thirteen decisions spanning gateway placement, the MCP estate, LLM serving, agent build stacks, identity, evaluation and observability — across Microsoft, Databricks, ServiceNow, SAP and Boomi.

One control plane

Many execution planes

Time-boxed experiments

What Ball asked, and how we have answered it

Thirteen questions in four groups. Each is answered as a decision with a recommendation, the rationale, the alternatives we rejected and the condition that would change our mind.

A Use cases the architecture must support

- SDLC / PDLC acceleration across ERP, integrations, Databricks, MDM and data cleanup
- Reconciliation between Databricks reporting and the source ERP
- Agents that monitor and improve data hygiene and governance

B Strategic direction

- The balance between innovation and standardization
- When Ball standardizes on approved platforms, and when it enables experimentation

C Tech stack and governance of agentic AI

- Four decisions to make in 2026: gateway, MCP hosting, MCP governance, LLM serving
- Four to design for now and decide in 2027: build stacks, evaluation, identity, observability

D Architecture workshop alignment

- Where our reference architecture agrees with the workshop material, where it differs, and why
- Addressed on slide 18 once the material is received

The test we applied to every question

- Centralize a decision only when decentralizing it creates a second source of truth
- Everything else is federated to where the data and the builders already are

Two things this response does not do

- It does not give cost or schedule estimates — those still follow the readiness assessment
- It does not pick 2027 products today; it sets the trigger that forces each choice

Items marked [CONFIRM] are assumptions about the Ball estate that we need validated before the recommendations become commitments. They are listed in full on slide 19.

Standardize the control plane. Federate the build plane.

Ball's estate is multi-vendor by strategy. A single-stack answer would be wrong; an ungoverned multi-stack answer would be expensive. This is the resolution.

ONE OF EACH

Control plane

- Identity and how it propagates
- The agent, tool and MCP registry
- The policy set: data classes, residency, redaction
- The trace schema and where traces land
- The evaluation contract and the gold sets
- The approved-model allow-list

Singular, because two of any of these creates a blind spot that cannot be reconciled after the fact.

MANY, BY DATA GRAVITY

Execution plane

- Azure AI Foundry for applications and agents
- Databricks for anything grounded in lakehouse data
- ServiceNow for employee and service workflows
- SAP and Boomi within their own process boundaries
- IDE and DevOps agents in the engineering estate

Plural, because forcing one runtime means moving Ball's data to it — which costs more than governing two.

TIME-BOXED

Experiment plane

- Any tool on the permitted list
- Non-production or synthetic data only
- No connection to a system of record
- No business process may depend on it
- Expires in 90 days unless promoted

Bounded, because the alternative is not zero experimentation — it is unmonitored experimentation.

The test to apply to any future platform question: if we allow two of this thing to exist, can we still answer — who did what, on whose behalf, with which data, at what cost, and was it any good? If no, it belongs in the control plane. If yes, federate it and stop arguing about it.

The eight platform decisions, at a glance

Sequenced by reversibility, not by urgency. Decisions that are expensive to unwind are made in 2026; decisions that are cheap to change are deliberately deferred to 2027 behind a stated trigger.

#	Decision	Recommendation	Decide	Reversibility
1	AI gateway placement	Two enforcement points — Azure APIM/Foundry for the enterprise, Databricks Mosaic AI Gateway inside the lakehouse — under one ServiceNow control-tower register.	2026	Low — traffic migration and key revocation are one-way
2	Hosting custom MCP servers	Central code and pipeline, near-to-domain runtime, central discovery. JDE beside AIS, SAP in its landing zone, Databricks tools via managed servers.	2026	Medium — placement can move, the auth pattern cannot
3	Governing allowed MCP servers	Central allow-list enforced at three points: catalogue registration, network egress, managed client configuration. Approval workflow in ServiceNow.	2026	Low — an ungoverned estate cannot be inventoried later
4	LLM serving	Two planes: Foundry for application and agent workloads, Databricks for lakehouse-grounded workloads. SaaS-embedded AI is a registered exception.	2026	High — workloads move if the equivalence table is kept
5	Agent build stacks	Three sanctioned lanes, at most two pro-code frameworks. Microsoft Agent Framework as default, LangGraph as the alternative.	2027	High — provided agents are defined as code
6	AI evaluation	Platform-native execution under one central contract. Ball owns the gold sets. A dedicated vendor only on a stated trigger.	2027	High — the gold sets are the asset and stay portable
7	Agent identity	One IdP, first-class agent workload identities, two identities on every action. Governing layer follows Ball's IdP. [CONFIRM]	2026 contract	Low — the identity model is the hardest retrofit
8	AI observability	Instrument once with OpenTelemetry, land twice — APM for operations, Databricks and MLflow for AI quality. One trace id.	2027	Medium — the span schema is sticky, the tools are not

Reversibility is the sequencing argument. Deciding items 1, 2, 3 and 7 late means retrofitting governance onto agents already in production — reliably the most expensive order to do this in.

Accelerating the SDLC and PDLC across the Ball estate

ERP development, integrations, Databricks engineering, MDM and data cleanup. Covered by the proposal — with one addition we recommend making explicit in the final deliverable.

RECOMMENDATION

- Treat design-time metadata as a first-class tool surface: the MCP servers must expose object dictionaries, definitions and configuration — not only runtime execution — because that is what an agent needs to author ERP change correctly.
- Ship AI-assisted change through the existing harness rather than a parallel process: provenance tagging, the golden pipeline, policy gates, and no unattended merge to a protected branch.

Design-time context packs to build

- JDE — object and table dictionary, ER definitions, custom object inventory, orchestration definitions, UDCs
- SAP — ABAP repository objects, CDS views, config and transport history
- Boomi — process and component library, connector configs, existing maps
- Databricks — Unity Catalog schemas, lineage, job and workflow definitions
- MDM — domain models, match rules, survivorship logic

Already in the proposal

- Harness engineering and the golden pipeline — slides 22 and 23
- MCP server 4 over repositories, pipelines, issues and deployments — slide 34
- DE.AI IDE ↔ Databricks integration with identity and gateway routing — slide 29
- Agent DLC: agents versioned, promoted and rolled back as artefacts — slide 33

How Ball will know it worked

- DORA metrics split by AI-assisted and human-authored change — not a blended number
- Change-failure rate must not rise as throughput rises; that is the whole test
- Rework rate on generated ERP objects and integrations
- Share of repositories building through the golden template

The failure mode to avoid: measuring adoption of the assistant instead of the delivery outcome. Licences deployed and prompts issued tell Ball nothing about whether change reaches production faster or more safely.

Does my Databricks report reconcile with the source ERP?

A reference pattern, and the clearest statement of the hybrid rule: the reconciliation is computed deterministically; the agent explains the break, it never decides whether the numbers match.

01

Define

A reconciliation control table fixes the grain, the measure, the tolerance and the schedule. Approved by Finance, versioned like code.

02

Compute

A deterministic job reads both sides — the ERP through its MCP read tools, the lakehouse through SQL — and records the break. No model involved.

03

Triage

The agent classifies the cause: timing and late arrival, transformation defect, master-data mismatch, semantic or filter difference, currency or UOM, source-side correction.

04

Evidence

It gathers the supporting records from both systems, cites lineage back to the table and the transaction, and scores its own confidence.

05

Resolve

Output is an explanation with citations plus a proposed fix and a ticket. A named steward approves; the fix executes through the normal change path.

Prerequisites — and they are real

- Canonical semantic definitions: 'net sales' must mean one thing before anything can reconcile to it
- ERP read tools at the same grain as the report, not at summary level
- A gold set of past known breaks, so the classifier can be evaluated rather than trusted

The rules that make it approvable

- Tolerances are policy set by Finance, never a model judgement
- The agent asserts no match — it explains a difference the deterministic job already found
- Every claim carries lineage to a table and a source transaction, or it is not shown

We recommend this as a lighthouse candidate alongside QA regression: it is bounded, its success is measurable in break count and mean time to explain, and it builds the trust in the Databricks estate that every later agent depends on.

Agents that monitor and improve data hygiene and governance

The same hybrid rule, applied to master data: continuous deterministic checks find the problem, the agent explains and proposes, a steward disposes, and the write goes through the governed path.

1 What the agents watch

- Duplicate and near-duplicate records across customer, supplier and item domains
- Missing, invalid or out-of-contract attributes on records feeding agents and reports
- Classification and tagging gaps in Unity Catalog
- Stale and orphaned assets, ownership gaps, freshness and SLO breaches
- Policy drift: assets whose classification no longer matches their content

2 What they are allowed to do

- Propose a merge, a correction or a classification — with the evidence attached
- Draft the steward work item, pre-populated and prioritised by impact
- Never write to the MDM or ERP record directly; every change executes through the approved orchestration or service
- Escalate on low confidence rather than guessing

3 How quality is proven

- A gold set of past adjudicated duplicates and corrections
- False-positive rate tracked as a first-class metric — steward trust is lost at the third bad proposal
- Acceptance rate of proposals by domain
- Master-data error rate and downstream rework in the ERP estate

WHERE THIS SITS IN THE PROPOSAL

- This is the ERP master-data quality lighthouse recommended on slide 41, and it is the operational arm of the data contracts and reconciliation building blocks on slide 15.
- Sequencing note: run it after the reconciliation pattern, not before. Reconciliation exposes which master-data defects actually matter, so the hygiene backlog is prioritised by evidence rather than by what is easiest to detect.

Governance framing for Internal Audit: these agents change the cost of data stewardship, not the control over it. The steward's approval remains the control event, and it remains logged.

Innovation and standardization are not a trade-off to balance

They are two zones with a gate between them. Ball's problem is not too much of one — it is that nothing today defines how something moves from one to the other, or how it gets retired.

EXPLORE

Sandbox

- Any tool on the permitted list
- Non-production or synthetic data only
- No system-of-record connection
- No business dependency permitted
- Expires at 90 days

No committee. Speed is the point.

PROVE

Pilot

- Real data, limited named users
- Registered, with an owner
- Gateway-routed, identity propagated
- Passes the evaluation gate
- Time-boxed with a success measure

Can be killed without ceremony.

SCALE

Production

- Full ADLC and change control
- Named SLA and on-call owner
- Continuous evaluation and drift alerts
- Periodic re-certification
- Documented rollback and kill switch

Funded and supported as platform.

Promotion triggers — written down

- Two or more domains need the same capability
- It touches a system of record or regulated data
- It exceeds an agreed spend or user threshold

Sunset triggers — equally written down

- Unused for two quarters
- Superseded by an approved standard
- Fails re-certification, or the vendor withdraws support

The list has four tiers, reviewed quarterly

- Standard — default, funded, supported
- Permitted — allowed with stated conditions
- Contained — sandbox only
- Prohibited — blocked at the network

For Ball specifically: standardization cannot mean one vendor, because the estate is deliberately multi-vendor. It has to mean one set of interfaces — which is the subject of slide 17.

Where the AI gateway really falls — and how many Ball needs

Ball's question names three products that are not the same kind of thing. Separating what each one actually does resolves most of the argument before any choice is made.

RECOMMENDATION

- One policy enforcement point per unavoidable execution boundary — for Ball that is exactly two, Azure and Databricks — with ServiceNow AI Control Tower as the single register and risk workflow over both. Do not buy a third gateway, and do not ask one product to do all three jobs.

JOB A

Model egress control

Sits on the network path between an agent and a model. Enforces routing, keys, spend caps, redaction, injection defence and full capture.

Azure API Management AI Gateway / Foundry

JOB B

In-platform model access

The same job, inside a data boundary that governed data must not leave. Enforces without exporting the lakehouse.

Databricks Mosaic AI Gateway

JOB C

AI control tower

Not on the data path. Inventories AI assets and carries risk tiering, approval, lifecycle and compliance evidence.

ServiceNow AI Control Tower

Interoperability with non-Microsoft stacks is a contract, not a product

- Every gateway must carry an Entra-issued token with end user, agent id and cost centre; enforce the same model allow-list and redaction policy; emit OpenTelemetry GenAI spans to one store; and report spend to one view. A gateway that cannot meet all four does not get production traffic.

Where SaaS-embedded AI cannot be routed

- NowAssist, SAP Joule, Boomi AI, GitHub Copilot and Genie ship the model inside the product. They are governed by contract, tenant configuration and registration in the control tower as accepted exceptions with named owners — not by pretending the gateway covers them.

Where to host Ball's custom MCP servers

'Local, near-to-domain or central' is three questions wearing one coat. Each has a different right answer, and answering them separately removes the deadlock.



Where the code lives

Central

- One repository template and one golden pipeline for every server
- Ownership by the source-system team via CODEOWNERS, not by the AI hub
- Steps 5 to 8 of the provisioning sequence become platform capability, run identically each time



Where the process runs

Near-to-domain

- JDE server in the ERP zone beside AIS; SAP server in the SAP landing zone or on BTP
- Latency and chattiness: agents make many small calls, and a long network path shows up in every one
- Circuit breaking protects the source — an agent retry storm against AIS is a genuine outage risk
- The team that gets paged owns the process



Where it is discovered

Central

- One enterprise index lists every server regardless of where it runs
- Azure API Center as the index of record; Unity Catalog governs lakehouse-resident tools
- An agent should never need to know which network zone a tool lives in

ON THE TWO CASES BALL NAMED — JDE AND SAP API CREATION

- These are two different needs and both matter: exposing existing JDE orchestrations and SAP services as governed tools, and helping engineers author new ones. The second requires the server to expose design-time metadata as read tools, not only runtime execution.
- Local — a developer's machine running a server over stdio — is a prototyping tool. It carries no audit, no identity propagation and no rate limit. Permit it in the sandbox zone against non-production data; block it from production-connected clients by managed policy.

Where to govern which MCP servers are allowed

Central, and enforced in three places — because a catalogue nobody is forced to consult is documentation, not governance.

RECOMMENDATION

- One central allow-list, enforced at three points: registration in the catalogue (Azure API Center as the enterprise index of record, Unity Catalog for lakehouse-resident tools), network egress from the agent runtimes, and managed client configuration pushed to IDEs, Copilot Studio and Foundry so a developer cannot add an arbitrary server to a production-connected client.
- The request, risk-tiering and approval workflow lives in ServiceNow, where Ball's change and risk process already runs. The catalogue holds the record; ServiceNow holds the decision.

Rationale

- Enforcement has to sit on the path, not in a policy document — the network allow-list is what makes the catalogue true
- Client configuration is the gap most organisations miss: the server estate is governed while the clients quietly are not
- Using ServiceNow for approval avoids inventing a second change process for AI
- One index means an auditor gets a single answer to 'what tools can reach our ERP'

What we are not recommending

- Per-domain allow-lists — they produce an estate nobody can inventory, and the exposure is cross-domain by nature
- A catalogue with no enforcement point behind it
- Trusting third-party MCP servers on reputation: pin versions and image digests, no runtime fetch from public registries, and re-review tool descriptions on every version change because a changed schema is a prompt-injection surface
- Granting one server both untrusted-content read and privileged write in the same session

The registration record

- Owner, source system and business purpose
- Data classes in scope, and residency constraints
- Read or write, and the approval checkpoint for each write
- Authentication pattern — OAuth with end-user propagation, never a shared principal
- Version, rate limits, risk tier, evaluation record, expiry date

No unregistered server obtains a gateway credential or is reachable from an agent runtime. That single rule is what converts the register from an inventory into a control.

Where Ball should serve LLMs from — and the exceptions

One rule settles almost every case: serve the model next to the data that grounds it, and route everything that can be routed through a gateway.

PLANE 1

Azure AI Foundry

Default for application and agent workloads, and anything M365-adjacent.

- Model breadth including frontier models, with regional deployment control
- Entra-native identity, which is where Ball's application estate and access model already sit
- The natural home for Copilot Studio and Microsoft Agent Framework workloads

PLANE 2

Databricks model serving

Default whenever the grounding context is governed lakehouse data.

- No egress of governed data, and lineage from an answer back to the source table
- Vector and search indexes co-located with the tables they index
- Batch, streaming and fine-tuned small models for narrow high-volume tasks, under one set of controls

The accepted exceptions — govern them, do not fight them

- Genie, NowAssist, SAP Joule, Boomi AI and GitHub Copilot ship the model as part of the product and cannot be gateway-routed
- Governed instead by contractual data terms, tenant configuration, log export where offered, and registration with a named owner
- Boundary rule: embedded AI may operate on data already inside its own product. The moment it needs data from another Ball system, it comes through MCP and the gateway

What is prohibited, and one thing to maintain

- Direct vendor API keys held by teams, and consumer-tier tools touching Ball data — blocked at the network, per step 8 of the gateway plan
- Any model endpoint absent from the allow-list, including inside an approved platform
- Maintain a model equivalence table: for each task class, the approved model on each plane. It is what lets a workload move planes later without a re-architecture

[CONFIRM] Current Foundry and Azure OpenAI deployment state and regions, and the Databricks workspace and region topology, before this becomes a commitment on residency.

One agent build stack, or several?

Several — but three sanctioned lanes, not an open field, and at most two pro-code frameworks. The framework is not the governance boundary, which is what makes more than one affordable.

LANE A

In the flow of work

Copilot Studio • ServiceNow AI Agent Studio

- Built by business and service teams where the work already happens — Teams, M365, ITSM, HR
- Chosen because the data and the user are already inside the product
- Constrained to registered MCP tools; no direct system-of-record writes outside approved services

LANE B

Grounded in data

Databricks Agent Bricks • Mosaic AI Agent Framework

- Default for anything whose context is lakehouse data — retrieval, analytics, reconciliation, hygiene
- Keeps governed data and its lineage inside the boundary that already governs it
- The strongest native evaluation and tracing of the three lanes

LANE C

Pro-code orchestration

Microsoft Agent Framework • LangGraph as the alternative

- For multi-step, cross-domain work needing explicit control flow, supervision and hand-off semantics
- Microsoft Agent Framework as the default, given Ball's Azure and Entra centre of gravity
- LangGraph sanctioned where graph-level control is genuinely required. Cap the total at two

Not recommended as Ball standards

- CrewAI Enterprise and Topaz: evaluate only against a proven capability gap, and keep them in the sandbox zone until one is demonstrated. Every additional framework multiplies the identity, evaluation and observability integration work, and leaves a hiring and maintenance tail long after the enthusiasm has moved on.

What makes three lanes affordable

- Whichever lane builds it, an agent registers in the same registry, propagates the same identity, emits the same traces and passes the same evaluation gate. Define agents as code — prompt, context pack, tool manifest, eval suite — so a lane change is a port rather than a rebuild, and avoid framework-specific memory stores.

AI evaluation — platform-native, or a dedicated stack?

Platform-native execution under one central contract, with Ball owning the gold sets. A dedicated vendor is a 2027 decision behind a stated trigger, not a 2026 purchase.

RECOMMENDATION

- Run evaluations where the agents run — MLflow and Mosaic AI Agent Evaluation on the Databricks side, Azure AI Foundry evaluations on the Foundry side, native tooling for closed platforms — and standardize the artefacts across all of them: gold sets, scorer definitions, thresholds and a common results schema, stored in Ball's own catalogue and version-linked to each agent release.
- The control is the CI gate, not the vendor: no agent is promoted unless its evaluation record passes the thresholds recorded against its version.

Rationale

- Native evaluation is closest to the traces, the retrieval and the data, which is where the diagnostic value is
- A common results schema gives comparability across lanes without a second product
- It defers a licence decision until Ball has enough production agents to specify what it actually needs
- It keeps evaluation inside the release pipeline, where it functions as a gate rather than a report

What we are not recommending

- Buying an evaluation platform to compensate for missing gold sets. The scarce asset is the labelled Ball question set, not the scoring interface — a tool without gold sets measures nothing
- Letting the vendor own the gold sets. They are proprietary Ball knowledge assets: they live in Ball's repository and catalogue, and they stay exportable
- Per-team evaluation approaches, which make two agents' scores incomparable

Adopt a dedicated stack when

- Two or more build stacks are in production and Ball needs one comparable score across them
- Quality scoring is needed online as a runtime guardrail, not only pre-release
- Audit evidence requirements exceed what native tooling can produce
- Evaluation engineering time crosses an agreed threshold
- Then: a 2027 bake-off — Galileo, Arize, Braintrust — against exit criteria agreed in advance

The 2026 work that makes the 2027 decision easy: build the gold sets per domain now, as part of the readiness assessment. They are required either way, and they are the input to any bake-off.

Agent identity — platform-native, or a governing stack?

One identity provider, first-class agent workload identities, and two identities on every action. Agree the contract in 2026; the product that implements it is replaceable.

RECOMMENDATION

- One identity provider — Entra ID [CONFIRM Ball's primary IdP]. Every agent gets a first-class workload identity: no shared service principals, no personal keys, no long-lived automation accounts.
- Two identities on every action: the agent's own, plus the acting user's carried on-behalf-of, so JD Edwards, SAP and Unity Catalog authorization stay authoritative. That single rule is what makes least privilege real rather than declared.

Rationale

- Governing layer follows the IdP: Entra Agent ID and Agent 365 for registry, agent joiner-mover-leaver, conditional access and revocation. Databricks, ServiceNow and SAP agent identities federate to it
- Identity is the hardest thing to retrofit: agents built on shared principals cannot be re-identified later without rebuilding every tool grant
- On-behalf-of means Ball builds no second permission model — the source systems keep theirs
- Agent access reviews become the same process as for people, run by the same team

What we are not recommending

- Two agent-identity governing planes. If Okta is Ball's primary IdP the recommendation pivots to Okta with workload identities federated to it — but never both
- Waiting for the product category to mature before enforcing the contract. The contract can be enforced today with workload identities and on-behalf-of flows
- Standing broad privileges of any kind, including 'temporary' ones granted during a pilot

Non-negotiables, whichever product wins

- Short-lived tokens; entitlements resolved at call time, not build time
- Least-privilege tool visibility, scoped per agent
- Every action attributable to agent version, acting user, tool and data scope
- On decommission, identity, tokens, tool grants and secrets revoked; the audit trail retained
- Human approval stays a distinct, logged event — an agent never approves its own write

[CONFIRM] Ball's primary identity provider and whether agent workload identity is in scope for the current Entra roadmap. This is the single assumption that most changes the recommendation.

AI observability — platform-native, or a dedicated stack?

Both, for different questions. Instrument once with OpenTelemetry and land the traces twice: the APM answers what broke, the AI-native tooling answers why the answer was wrong.

RECOMMENDATION

- One instrumentation standard: OpenTelemetry GenAI semantic conventions, with a span model of session → agent → delegation → tool call → retrieval → model call → human checkpoint, carrying tokens, cost, latency and outcome on every span. The gateway emits at the boundary as the catch-all; runtimes propagate trace context through delegation.
- Land the traces twice: the enterprise APM — Dynatrace [CONFIRM Ball's standard] — as the operational pane, and Databricks and MLflow traces as the AI-quality pane for context capture, retrieval scores, evaluation linkage and lineage. One trace id spans both.

Rationale

- Agent failures are rarely model failures. Expired tokens, throttling, schema drift and source latency dominate, and those live in the APM beside the rest of the estate
- Generic APM cannot answer why a model saw what it saw. AI-native tracing can, because it captures the assembled context
- SRE alerting, on-call and incident process already exist in the APM — reusing them beats an AI-only incident path
- Instrumenting once means a runtime change is not a re-instrumentation project

What we are not recommending

- A third dashboard. Two panes with one trace id is the ceiling; anything more and nobody looks at any of them
- Per-application SDK instrumentation as the primary capture path — it guarantees blind spots wherever a team forgets
- Taking cost figures from the APM. Spend attribution is authoritative at the gateway, where the request is priced

The differentiator to insist on

- Capture the assembled context, not only the model call: which documents were retrieved, their scores, and what was evicted under the token budget
- Without it Ball can see that an agent was wrong and never establish why
- It is what makes context packs improvable from evidence rather than intuition
- This extends slide 37 of the proposal, and is why the AI-quality pane cannot be dropped

Sequencing: the span schema is agreed in 2026 alongside the gateway, because traces not captured in the right shape cannot be reconstructed later. The tooling choice can wait.

Six interfaces every platform must honour

This is what 'a cohesive enterprise AI strategy rather than a collection of platform-specific recommendations' looks like when written down. A platform Ball buys in 2028 is adopted by meeting these six.

01 Identity

An Entra-issued token carrying the end user, the agent identity and a cost centre, propagated on-behalf-of to the source system, which keeps its own authorization.

02 Tools

MCP, with typed schemas, registered in the enterprise catalogue, visible to an agent only under least privilege. Writes execute through approved deterministic services.

03 Policy

One policy set — data classes, residency, redaction, blocked topics, injection defence — enforced at each boundary and conformance-tested, not assumed.

04 Telemetry

OpenTelemetry GenAI spans plus assembled-context capture, exported to the enterprise store under a single trace id per run.

05 Evaluation

A gold set, scorers and thresholds held in Ball's catalogue, version-linked to the agent release and enforced as a promotion gate in CI.

06 Registration

An entry in the AI register: owner, purpose, data scope, tools, risk tier, evaluation record, SLA and expiry. No entry, no credential.

Ball cannot standardize on one vendor — the estate is multi-vendor by strategy. It can standardize on these six, and that is the only version of standardization that survives the next platform Ball buys.

Alignment with Ball's architecture workshop material

Ball's note references an attached workshop covering AI-assisted software development, MCP servers, identity management, governance and AI-enabled DevOps. The material has not yet reached us — this slide states how we will assess it.

1 What we will assess it against

- The six interfaces on the previous slide, one by one
- The five topics the workshop covers, mapped onto the reference architecture layers
- The eight decisions in this register — does the workshop assume a different answer to any of them?
- The hand-off rule: does anything in it let an agent become the system of control?

2 What we will report back

- Agrees — concepts already carried in the proposal, with the slide reference
- Differs — where we would recommend something else, and the rationale
- Absent — what the workshop covers that the proposal does not, and whether it should be added
- Open — decisions the workshop leaves unresolved that Ball still needs to make

3 Where we already expect strong overlap

- MCP as the tool protocol and the governed tool surface over systems of record
- Identity propagation and the rejection of shared service principals
- AI-assisted development governed through the existing delivery pipeline rather than beside it
- Governance enforced in the runtime rather than by review board

THE ONE THING WE WILL LOOK FOR HARDEST

- Whether the workshop treats governance as a gate applied to AI work, or as infrastructure that AI work runs on. The proposal takes the second position deliberately — steps 2, 5, 7 and 8 of the action flow are platform capability, which is why they belong to a division rather than to each project.
- If the workshop assumes the first, that is the substantive difference worth an hour of the working session, and it is a decision for Ball rather than a correction to either document.

Request: send the workshop material as a pre-read. We will return a written alignment analysis against these four categories before the session, so the time is spent on the differences rather than on establishing them.

The working session, and what changes in the final deliverables

We agree a working session is the right forum. We suggest running it as a decision meeting rather than a discussion, with this addendum as the pre-read.

1 Proposed agenda — 90 minutes

- 10 min — the organising principle, and agreement on the test
- 35 min — the four 2026 decisions: gateway, MCP hosting, MCP governance, LLM serving
- 20 min — the four 2027 decisions: agree the triggers, not the products
- 15 min — workshop alignment, on the material once received
- 10 min — updates to the final deliverables and owners

2 What we will add to the final deliverables

- A decision register appendix — the eight decisions as signed architecture decision records
- The six-interface conformance contract as a standing admission test for new platforms
- The reconciliation and data-hygiene reference patterns as worked examples
- Design-time context packs named explicitly in the SDLC workstream
- The three-zone innovation model folded into the governance section

3 What we need from Ball [CONFIRM]

- The architecture workshop material, as a pre-read
- Primary identity provider, and whether agent workload identity is on the Entra roadmap
- The enterprise APM standard
- Azure Foundry and APIM footprint, and Databricks workspace and region topology
- Boomi's intended role in the target state — integration runtime, tool surface, or both

THE OUTPUT WE WOULD LIKE FROM THE SESSION

- Eight decisions recorded, each with a named owner and a decide-by date — four ratified now, four with their triggers agreed. Everything else in the engagement can then be built against a settled architecture rather than a set of options.
- As in the proposal, no cost or schedule estimates are offered here by design. Both still follow the readiness assessment; committing to them earlier would be a guess dressed as a plan.